3643. Flip Square Submatrix Vertically

Solved

Easy | Topics | Companies | Hint

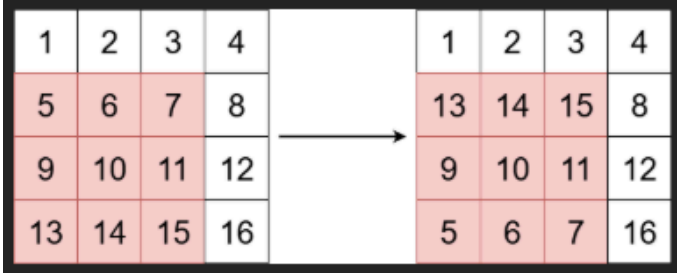
You are given an `m x n` integer matrix `grid`, and three integers `x`, `y`, and `k`.

The integers `x` and `y` represent the row and column indices of the **top-left** corner of a **square** submatrix and the integer `k` represents the size (side length) of the square submatrix.

Your task is to flip the submatrix by reversing the order of its rows vertically.

Return the updated matrix.

Example 1:



Input: `grid = [[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]], x = 1, y = 0, k = 3`

Output: `[[1,2,3,4],[13,14,15,8],[9,10,11,12],[5,6,7,16]]`

Explanation:

The diagram above shows the grid before and after the transformation.

Example 2:

> **Input:** `grid = [[3,4,2,3],[2,3,4,2]], x = 0, y = 2, k = 2`
>
> **Output:** `[[3,4,4,2],[2,3,2,3]]`
>
> **Explanation:**
>
> The diagram above shows the grid before and after the transformation.

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 50`
- `1 <= grid[i][j] <= 100`
- `0 <= x < m`
- `0 <= y < n`
- `1 <= k <= min(m - x, n - y)`

Python:

```
class Solution:
    def reverseSubmatrix(self, grid: List[List[int]], x: int, y: int, k: int) -> List[List[int]]:
        for i in range(k):
            for j in range(k // 2):
```

```
            grid[x + j][y + i], grid[x + k - j - 1][y + i] = grid[x + k - j - 1][y + i], grid[x + j][y + i]
        return grid
```

JavaScript:

```
const reverseSubmatrix = (grid, x, y, k) => {
    for (let i = 0; i < k; i++)
        for (let j = 0; j < k >> 1; j++)
            [grid[x + j][y + i], grid[x + k - j - 1][y + i]] =
                [grid[x + k - j - 1][y + i], grid[x + j][y + i]];

    return grid;
};
```

Java:

```
class Solution {
    public int[][] reverseSubmatrix(int[][] grid, int x, int y, int k) {
        for (int i = 0; i < k; i++) {
            for (int j = 0; j < k >> 1; j++) {
                int temp = grid[x + j][y + i];
                grid[x + j][y + i] = grid[x + k - j - 1][y + i];
                grid[x + k - j - 1][y + i] = temp;
            }
        }

        return grid;
    }
}
```